MELBOURNE SENSORY-AWARE TRAVEL

Technical Documentation

FIT5120 • Building Sensory-Friendly Urban Futures
UN Sustainable Development Goal 11

Document field	Value
Product name	CalmRoute Melbourne / Melbourne Sensory-Aware Travel
Document version	1.0
Date	9 August 2026
Team	TP12
GitHub Organization	https://github.com/FIT5120-Team12/TP12-Project_First/tree/main
Frontend public URL	https://melbourne-sensory-travel-team12.vercel.app/
Backend service URL	https://melbourne-sensory-api-team12.onrender.com

Document status This document describes the current review build and the selected production deployment. Public URLs and final regression results must be updated after deployment.

Contents

1. Introduction and Scope
2. Technical Requirements Traceability
3. Technology Stack
4. System Architecture
5. Frontend Architecture
6. Backend Architecture
7. REST API Documentation
8. Data Sources and Processing
9. Core Business Rules
10. Deployment Architecture
11. Security and Privacy
12. Quality Assurance and Acceptance Testing
13. Known Limitations
14. Local Development and Reproduction
15. References

Appendix A. Final Review and Deployment Checklist

1. Introduction and Scope

Melbourne Sensory-Aware Travel is a full-stack web application that supports sensory-sensitive commuters travelling through Melbourne CBD. The product focuses on pedestrian activity, lower-stimulation route comparison, historical activity patterns, and nearby public places that may provide an option to pause during a journey.

The current build is an onboarding MVP intended for technical review and acceptance testing. It uses cleaned City of Melbourne pedestrian, sensor-location and landmark datasets. The application separates the frontend and backend so that user-interface responsibilities, business rules and data access remain clearly defined.

Scope boundary The current route geometry is a prototype polyline produced by the backend. It is not turn-by-turn street navigation. Packaged pedestrian data is a historical snapshot and is not presented as live current conditions.

1.1 Primary technical objectives

- Provide a publicly deployable web solution for the onboarding topic.
- Keep frontend and backend responsibilities separate.
- Use open city data for data-driven route and activity features.
- Provide clear REST API contracts between Vue and Spring Boot.
- Implement observable acceptance-criteria states including High, Low, Data unavailable, peak-hour, lower-stimulation alternatives, prediction and nearby places.
- Keep limitations visible so the product does not overstate data freshness or route accuracy.

2. Technical Requirements Traceability

The technical mentor guidance emphasises a data-driven feature, separate frontend and backend development, team code management through a GitHub Organization, protection of sensitive information, and deployment to a publicly accessible URL rather than localhost-only delivery.

Technical requirement	Project response	Status
Data-driven feature	Cleaned pedestrian counts, sensor locations and landmarks are used by the backend.	Implemented
Separate frontend and backend	Vue SPA communicates with Spring Boot through REST/JSON.	Implemented
GitHub Organization	Repository should be managed through the team organization.	Team setup / verify before review
No sensitive information in GitHub	Secrets and deployment-specific values are environment variables.	Required before every push
Public deployment	Selected production plan is Vercel frontend + Render backend.	Pending final URLs
Open datasets	City of Melbourne pedestrian and landmark datasets are the data basis.	Implemented with cleaned snapshot

3. Technology Stack

Layer / tool	Technology	Purpose
Frontend framework	Vue 3	Single-page user interface
Frontend language	TypeScript	Typed application and API contracts
Build tool	Vite	Development server and production build
Routing	Vue Router	Page navigation
State management	Pinia	Cross-page application state

HTTP client	Axios	REST communication with Spring Boot
Map	Leaflet + OpenStreetMap tiles	Route and location visualisation
Styling	Tailwind utilities + project CSS	Responsive interface styling
Backend framework	Spring Boot	REST API and business logic
Backend language	Java 21	Server implementation
Build system	Maven / Maven Wrapper when committed	Dependency and package management
Runtime data	Cleaned CSV snapshot	Current MVP data-access layer
Frontend hosting	Vercel	Selected public frontend hosting
Backend hosting	Render	Selected public Spring Boot hosting
Version control	Git + GitHub Organization	Team collaboration and review

4. System Architecture

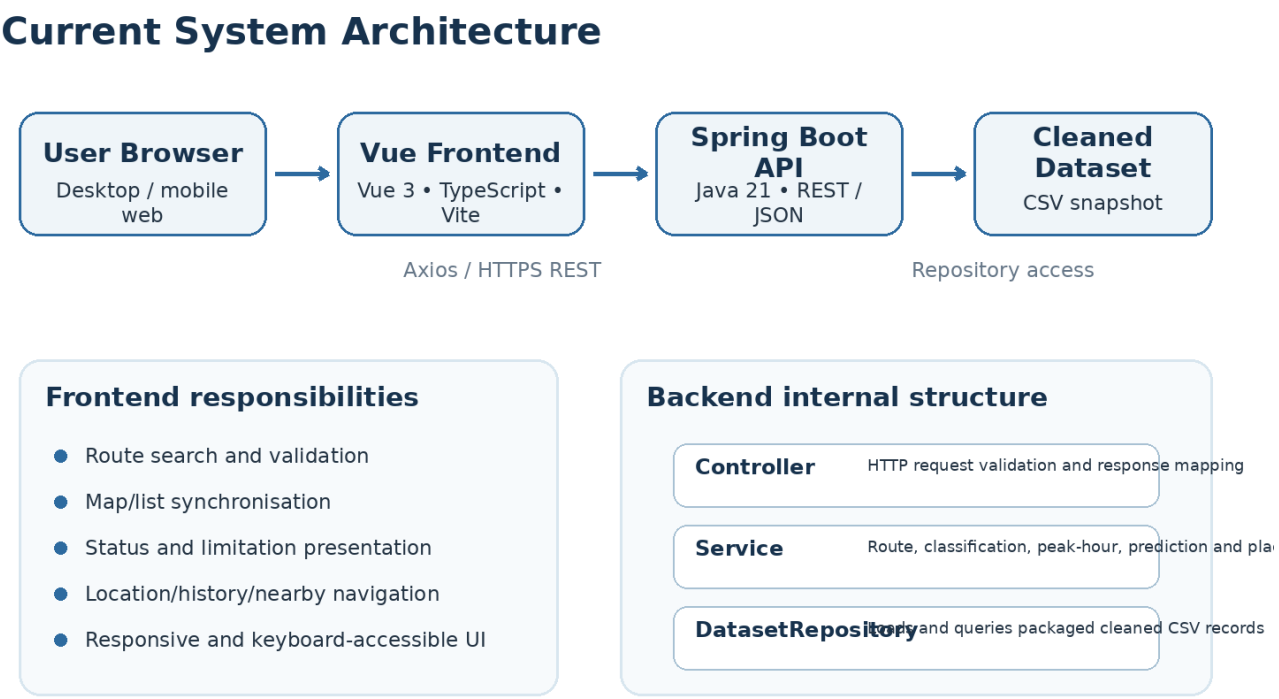

Figure 1. Current application architecture.

The application uses a client–server architecture. Vue is responsible for user interaction and presentation. Spring Boot owns validation, route-result preparation, pedestrian aggregation, sensory classification, historical calculations and nearby-place filtering. The backend loads the cleaned dataset into memory through DatasetRepository when the application starts.

4.1 Architectural boundary

- Frontend does not directly read CSV files.
- Frontend does not calculate production High/Low classification in HTTP mode.
- Frontend does not calculate the historical prediction from raw readings.
- Backend returns route coordinates and status values; Leaflet renders the geometry.
- The current DatasetRepository can later be replaced by a database-backed implementation while keeping REST DTOs stable.

5. Frontend Architecture

The frontend is a Vue 3 single-page application. Components and views consume data through a service layer rather than querying datasets directly. Pinia stores only state that must persist across screens, such as selected route search values and current selection.

```
frontend/
├── src/
│   ├── api/
│   ├── components/
│   ├── layouts/
│   ├── router/
│   ├── stores/
│   ├── styles/
│   ├── types/
│   └── views/
├── package.json
└── vite.config.*
```

5.1 Frontend responsibilities

- Origin, destination and departure-time input.
- Client-side input feedback and asynchronous loading/error states.
- Route list and Leaflet map synchronisation.
- Visible text presentation for High, Low and Data unavailable.
- Peak-hour and prediction context presentation.
- Location detail, historical trend and nearby-place navigation.
- Responsive layout, keyboard focus and non-colour-only status communication.

5.2 HTTP service boundary

Views call service functions that use Axios in the integrated build. This keeps endpoint details outside page components and makes the frontend easier to maintain when backend field names or hosting URLs change.

6. Backend Architecture

The backend follows a layered Spring Boot structure. HTTP responsibilities are separated from business rules and dataset access. Constructor injection is used so that Spring manages dependencies between controllers, services and repositories.

```
backend/src/main/java/...
├── controller/
│   ├── HealthController
│   ├── RouteController
│   ├── LocationController
│   └── GlobalExceptionHandler
├── service/
│   ├── RouteService
│   ├── LocationService
│   ├── HistoryService
│   └── NearbyPlaceService
├── repository/
│   └── DatasetRepository
├── model/
├── dto/
```

```
├── config/
└── exception/
```

Backend area	Responsibility
Controller	Receives HTTP requests, validates request structure and returns DTOs / error responses.
Service	Implements route, classification, peak-hour, historical prediction, historical trend and nearby-place rules.
Repository	Loads and queries cleaned sensor, pedestrian and landmark CSV records.
Model	Represents internal dataset records.
DTO	Defines the frontend–backend request and response contract.
Exception handling	Converts application errors into predictable HTTP responses without exposing implementation details.

6.1 Dependency injection example

```
public RouteController(RouteService routeService) {
    this.routeService = routeService;
}
```

The controller does not construct RouteService directly. Spring creates the service bean and injects it through the constructor, reducing coupling and making responsibilities explicit.

7. REST API Documentation

Method	Endpoint	Purpose
GET	/api/health	Check whether the backend is running.
GET	/api/locations/supported	Return supported sensor-backed locations.
POST	/api/routes	Generate route options and route status context.
GET	/api/locations/{id}	Return location activity, freshness and contextual information.
GET	/api/locations/{id}/history	Return historical hourly trend data and summary.
GET	/api/locations/{id}/nearby-places	Return up to three eligible nearby public places.

7.1 Route request example

```
POST /api/routes
Content-Type: application/json

{
  "originId": "5",
  "destinationId": "3",
  "departureTime": "2025-10-21T17:30:00"
}
```

The route response contains route identity, estimated time and distance, geometry coordinates, sensory status, pedestrian average or unavailable state, lower-stimulation-alternative context, peak-hour context, historical prediction information and limitation/source metadata.

7.2 Health check

```
GET /api/health

{
  "service": "sensory-aware-backend",
  "status": "UP"
}
```

The health endpoint has been verified locally in the current development environment.

8. Data Sources and Processing

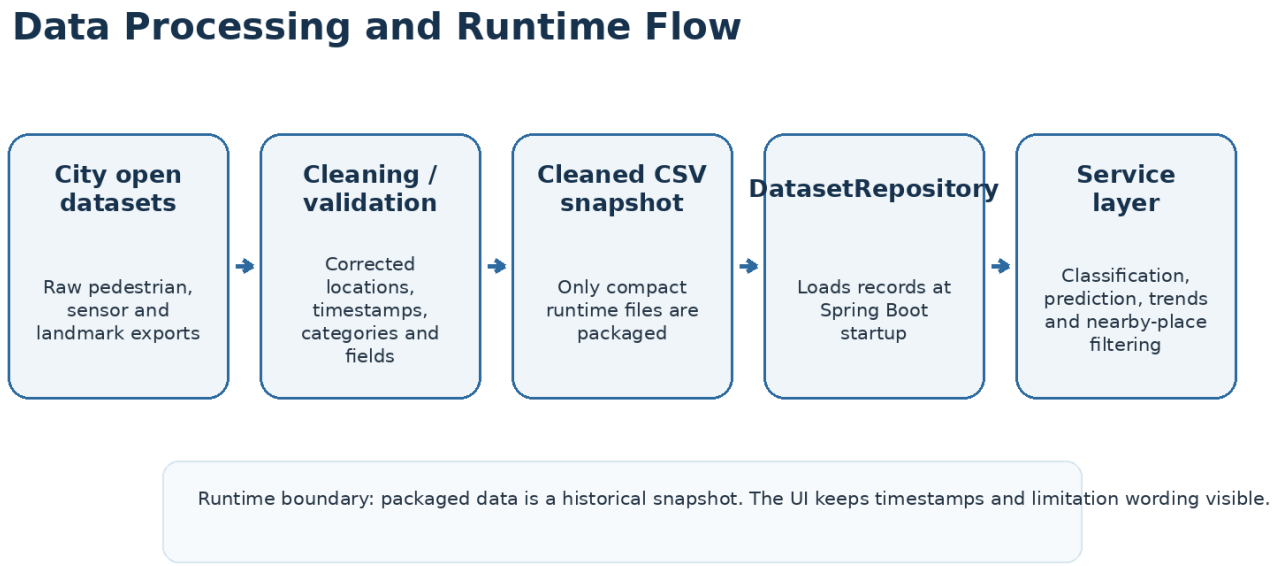

Figure 2. Dataset preparation and runtime flow.

Packaged file	Rows	Runtime / project use
sensor_location.csv	6	Supported CBD sensor locations and coordinates
pedestrian_hour_count.csv	8,052	Route classification, peak-hour evaluation, prediction and historical trend
pedestrian_minute_count.csv	18,589	Latest available location activity
landmark.csv	38	Nearby public places
landmark_category.csv	2	Category reference
candidate_sensor_locations.csv	14	Audit evidence; not queried by runtime API

8.1 Data ranges and freshness

- Hourly cleaned data: 15 September 2025 to 9 November 2025.
- Minute cleaned data: 2 August 2026 23:55 (+10:00) to 6 August 2026 11:55 (+10:00).

Because the sources cover different time windows, the packaged dataset must not be described as a live current-state feed. Location Detail retains the source timestamp and presents freshness/limitation information.

8.2 Open data basis

- City of Melbourne Pedestrian Counting System – Past Hour (Counts per Minute).
- City of Melbourne Pedestrian Counting System – Sensor Locations.
- City of Melbourne Pedestrian Counting System – Monthly / hourly pedestrian counts.
- City of Melbourne Landmarks and Places of Interest.

9. Core Business Rules

9.1 Route sensory classification

Condition	Displayed status
Average pedestrian activity > 60 pedestrians/minute	High
Average pedestrian activity ≤ 60 pedestrians/minute	Low
No usable pedestrian sensor data	Data unavailable

Hourly totals are converted to pedestrians per minute by dividing the hourly total by 60. The average count and threshold context are presented with the status.

Data-backed review example Princes Bridge sensor 5: 5,511/hour = 91.85/min → High. Melbourne Central sensor 3: 3,065/hour = 51.08/min → Low.

9.2 Lower-stimulation alternative

When route options include at least one High route and at least one Low route, the Low route can be identified as a lower-stimulation alternative. The wording must remain limited to available pedestrian data and must not describe a route as guaranteed quiet, safest or best.

9.3 Peak-hour rule

- Melbourne weekdays 07:00–10:00.
- Melbourne weekdays 16:00–19:00.

Peak-hour requests use the same High / Low / Data unavailable thresholds as other route requests. The result includes the limitation that actual conditions may differ from available pedestrian data and historical patterns.

9.4 Historical next-hour prediction

A route is eligible for the next-hour historical prediction only when at least four comparable readings are available from the same sensor, the same hour of day, and the same day type (weekday or weekend). The predicted value is the average of those comparable historical readings.

Predicted average	Displayed prediction status
> 60 pedestrians/minute	Higher pedestrian activity likely
≤ 60 pedestrians/minute	Lower pedestrian activity likely
Fewer than four comparable readings or no valid sensor	Prediction unavailable

9.5 Nearby public places

- Eligible categories: Library, Park, Garden and Reserve.
- Maximum three results; if only one or two are available, show only those results.
- Each result displays name, category and approximate distance from the selected destination.
- The interface states that category-based suggestions have not been verified as quiet or sensory-friendly.

10. Deployment Architecture

The selected deployment separates frontend and backend hosting. The user receives one Vercel frontend URL. The browser then calls the Render backend through HTTPS REST requests.

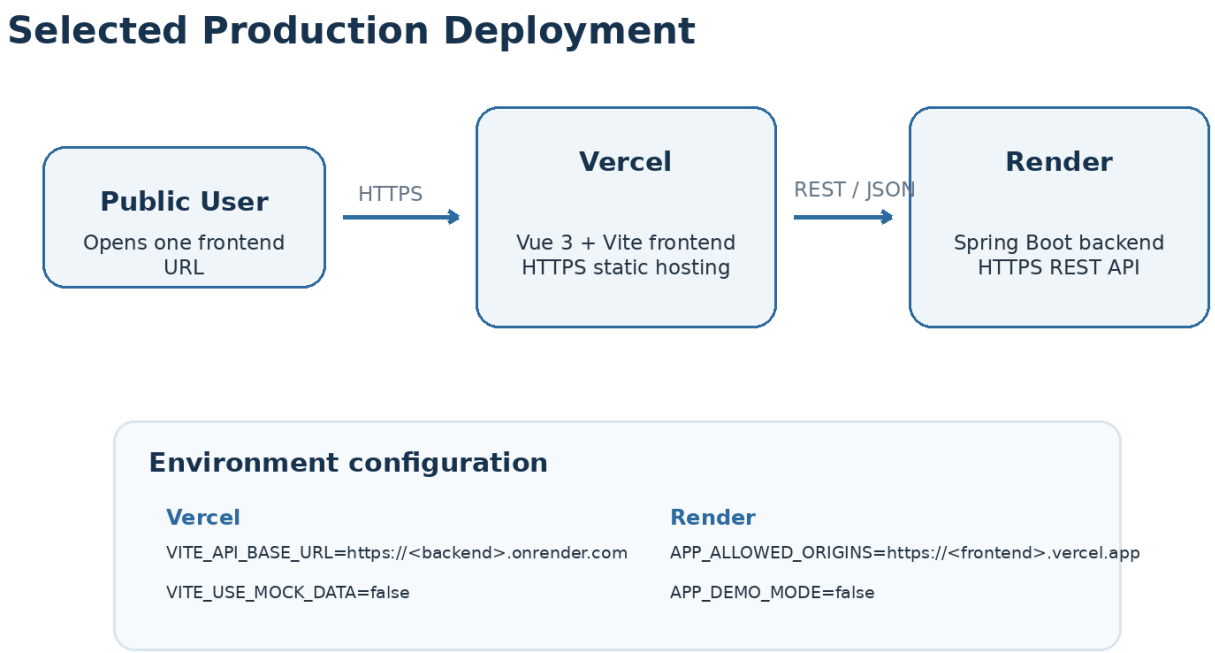

Figure 3. Selected Vercel + Render deployment.

10.1 Vercel frontend configuration

Setting	Value
Root directory	frontend
Build command	npm run build
Output directory	dist
VITE_API_BASE_URL	https://<backend>.onrender.com
VITE_USE_MOCK_DATA	false

10.2 Render backend configuration

Setting	Value
Root directory	backend
Runtime	Java 21 / Spring Boot
APP_ALLOWED_ORIGINS	https://<frontend>.vercel.app
APP_DEMO_MODE	false
Health endpoint	/api/health

Submission requirement Before final submission, replace the URL placeholders with the real deployment URLs and verify the site from a network environment that will be used for mentor review.

11. Security and Privacy

Control	Implementation / expectation
HTTPS	Use HTTPS for frontend and backend production traffic.
Environment variables	Do not commit secrets or environment-specific service URLs as credentials.
CORS	Backend should allow only approved frontend origins in production.

Input validation	Validate route request fields and unsupported locations before processing.
Error handling	Return clear errors without exposing internal stack traces or sensitive details.
Data privacy	Use aggregated pedestrian data and avoid unnecessary storage of user location history.
Repository hygiene	Exclude .env files, build output, dependency folders and IDE metadata as appropriate.

12. Quality Assurance and Acceptance Testing

Acceptance testing should distinguish visible user-interface evidence from backend logic evidence. A status appearing on screen is not by itself proof that the calculation is correct; the API response and relevant dataset rule should also be checked.

Acceptance area	Evidence to verify	Current status
Route information	Route path, estimated time and distance are visible.	Implemented; geometry remains prototype
High / Low / unavailable	Text status is shown and threshold context is visible.	Implemented; final UI consistency regression required
Lower-stimulation alternative	Low alternative is shown when High and Low routes coexist.	Implemented
No alternative	Warning plus Continue / Return actions can be demonstrated.	Implemented review state
Peak-hour	Weekday peak windows and shared threshold rule are supported.	Implemented
Nearby transport	At most one connection with name/type/distance.	Prototype metadata; destination-context verification required
Nearby places	Approved categories, maximum three, name/category/distance and limitation.	Implemented
Prediction	Eligibility, historical average, higher/lower status and unavailable state.	Implemented
Outside coverage	Unsupported location can return a recoverable state.	Implemented review state
Responsive / keyboard	Core flow should remain usable without colour-only meaning.	Final browser regression required

12.1 Required final regression before mentor acceptance

- Verify selected route status is identical on the map, route card and sensory summary.
- Verify destination details—not origin details—drive destination public-transport and nearby-place context where required by the approved criteria.
- Run the core flow in production mode with mock fallback disabled.
- Verify Vercel-to-Render API calls, CORS, HTTPS and nested Vue route refresh.
- Check browser console for blocking errors and confirm /api/health responds on the public backend.

13. Known Limitations

Limitation	Current boundary
Walking route geometry	Backend-generated prototype polylines are used instead of turn-by-turn pedestrian street routing.
Route-to-sensor association	Current prototype association is simplified; Route A uses the origin sensor and Route B uses the destination sensor.
Data freshness	Packaged pedestrian data is a historical snapshot, not a live current feed.
Public transport context	Nearby transport is contextual prototype metadata rather than a complete public-transport journey service.
Authentication	Sign-in is illustrative; guest access is the primary supported onboarding path.
Prediction	Next-hour status is a simple historical average and is presented as guidance rather than a guaranteed future condition.
Database	The current MVP uses packaged cleaned CSV files rather than PostgreSQL or Redis.

13.1 Future technical improvement path

- Introduce a RouteGeometryProvider backed by an approved walking-route provider.
- Spatially associate route geometry with sensors along the whole route rather than using simplified endpoint association.
- Move cleaned data into PostgreSQL when persistent or larger-scale queries justify the extra infrastructure.
- Add caching only when live/provider calls create measurable performance needs.
- Replace contextual transport metadata with an approved transport-data integration if the project scope requires it.

14. Local Development and Reproduction

14.1 Prerequisites

- Git
- Node.js 20+ and npm
- Java 21
- Maven 3.9+ or a committed Maven Wrapper

14.2 Start the backend

```
cd backend
mvn spring-boot:run

# If Maven Wrapper is committed on Windows:
.\mvnw.cmd spring-boot:run
```

Local backend health check: http://localhost:8080/api/health

14.3 Start the frontend

```
cd frontend
# Windows PowerShell
Copy-Item .env.example .env.local
npm install
npm run dev
```

Local frontend: http://localhost:5173

14.4 Local integrated environment

```
VITE_API_BASE_URL=http://localhost:8080
VITE_USE_MOCK_DATA=false
```

Production separation Production must not silently use the frontend mock fallback. The production Vercel environment should call the Render backend directly.

15. References

[1] FIT5120 Tech Mentor — Tech Requirements and Important Notes, Semester 2 2026.

[2] FIT5120 — Approved Acceptance Criteria: Sensory-Aware Route Planning and Sensory Environment Monitoring.

[3] Project README — Melbourne Sensory-Aware Travel FIT5120 Review Build.

[4] Project Architecture document — current Vue / Spring Boot / DatasetRepository architecture.

[5] Project Dataset Usage Audit — packaged files, row counts, date ranges and runtime use.

[6] City of Melbourne Open Data — Pedestrian Counting System: Past Hour Counts per Minute.

[7] City of Melbourne Open Data — Pedestrian Counting System: Sensor Locations.

[8] City of Melbourne Open Data — Pedestrian Counting System: Monthly Counts per Hour.

[9] City of Melbourne Open Data — Landmarks and Places of Interest.

Public dataset links

https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-past-hour-counts-per-minute

https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-sensor-locations

https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-monthly-counts-per-hour

https://data.melbourne.vic.gov.au/explore/dataset/landmarks-and-places-of-interest-including-schools-theatres-health-services-spor/information/

Appendix A. Final Review and Deployment Checklist

Complete this checklist before the technical mentor acceptance test and update the document fields on the cover page with the final URLs.

Check	Verification item	Done
Repository	GitHub Organization and repository URL are correct; no .env, secrets, node_modules, target or IDE-only files are committed.	finished
Frontend build	npm install and npm run build complete successfully in the final branch.	finished
Backend build	Spring Boot starts with Java 21 and /api/health returns status UP.	finished
Production mode	VITE_USE_MOCK_DATA=false and APP_DEMO_MODE=false for the normal public deployment.	finished
Vercel	Frontend loads from the final public HTTPS URL and direct page refresh works.	finished
Render	Backend is reachable through HTTPS and the frontend can call all required API endpoints.	finished
CORS	APP_ALLOWED_ORIGINS permits the exact Vercel production origin and is not unnecessarily broad.	finished
Core route flow	Supported origin/destination search displays route path, time, distance and consistent High/Low/Data unavailable text.	finished
Destination context	Destination details drive the required nearby transport and nearby-place flow.	finished
Prediction	Eligible, higher/lower and unavailable prediction states are verified against backend responses.	finished
Error states	Outside coverage, unavailable data and	finished

	API failure states are recoverable and do not display fabricated route data.	
Browser check	No blocking console errors; keyboard focus and text alternatives remain usable.	finished
Network check	Final public URL is tested from the Monash network or Monash VPN before demonstration.	finished